Case Study 7: Advanced Numeric Pattern Generator

1. Problem Statement

Develop a Python program that generates numeric patterns based on the number of rows entered by the user. The program should display multiple number patterns using loops and nested loops, helping users understand how numbers and spaces can be arranged in different ways.

2. Objectives

- To understand `for` loops and nested loops.
- To use variables and the `range()` function.
- To generate numeric patterns using simple logic.
- To understand how loop iterations control rows and columns.
- To improve logical thinking and problem-solving skills.

3. Assumptions

1. The user enters a positive integer representing the number of rows.
2. The program generates three patterns.
3. Pattern 1 prints increasing numbers in each row.
4. Pattern 2 prints the row number repeatedly.
5. Pattern 3 prints an inverted number pattern.
6. The program accepts row counts from 1 to 10.

4. Algorithm

1. Start the program.
2. Ask the user to enter the number of rows.
3. Validate that the number is between 1 and 10.
4. Use nested loops to generate Pattern 1.
5. Use nested loops to generate Pattern 2.
6. Use nested loops to generate Pattern 3 in reverse order.
7. Display all three patterns.

8. Stop the program.

5. Flowchart

Start
Enter number of rows
Is the input between 1 and 10?
Generate Pattern 1 using nested loops
Generate Pattern 2 using nested loops
Generate Pattern 3 in reverse order
Display all patterns
Stop

6. Pseudocode

```
None
START

INPUT number_of_rows

IF number_of_rows < 1 OR number_of_rows > 10:
    DISPLAY "Invalid number of rows"
ELSE:
    DISPLAY Pattern 1

    FOR i FROM 1 TO number_of_rows:
        FOR j FROM 1 TO i:
            PRINT j
        MOVE TO NEXT LINE

    DISPLAY Pattern 2

    FOR i FROM 1 TO number_of_rows:
        FOR j FROM 1 TO i:
            PRINT i
        MOVE TO NEXT LINE

    DISPLAY Pattern 3
```

```
    FOR i FROM number_of_rows DOWN TO 1:
        FOR j FROM 1 TO i:
            PRINT j
        MOVE TO NEXT LINE

STOP
```

7. Python Source Code

Save the file as `numeric_pattern.py`.

None

```
print("===== ADVANCED NUMERIC PATTERN GENERATOR =====")

try:
    rows = int(input("Enter the number of rows (1-10): "))

    if rows < 1 or rows > 10:
        print("Invalid input! Enter a number from 1 to 10.")

    else:
        # Pattern 1: Increasing numbers
        print("\nPattern 1: Increasing Numbers")

        for i in range(1, rows + 1):
            for j in range(1, i + 1):
                print(j, end=" ")
            print()

        # Pattern 2: Repeated row numbers
        print("\nPattern 2: Repeated Row Numbers")

        for i in range(1, rows + 1):
            for j in range(1, i + 1):
```

```
                print(i, end=" ")
            print()

        # Pattern 3: Inverted numbers
        print("\nPattern 3: Inverted Number Pattern")

        for i in range(rows, 0, -1):
            for j in range(1, i + 1):
                print(j, end=" ")
            print()

except ValueError:
    print("Invalid input! Please enter a whole number.")

print("\nPattern generation completed.")
```

8. Test Cases

Test case	Input	Expected result
1	4	All three patterns are displayed with four rows at most
2	1	Each pattern displays one number
3	10	All three patterns are displayed with up to ten numbers per row

4	`0`	Invalid number of rows message
5	`12`	Invalid number of rows message
6	`hell o`	Invalid input message

Sample output

None

```
===== ADVANCED NUMERIC PATTERN GENERATOR =====
Enter the number of rows (1-10): 4

Pattern 1: Increasing Numbers
1
1 2
1 2 3
1 2 3 4

Pattern 2: Repeated Row Numbers
1
2 2
3 3 3
4 4 4 4

Pattern 3: Inverted Number Pattern
1 2 3 4
1 2 3
1 2
1

Pattern generation completed.
```

9. Output Screenshots

```
print("===== ADVANCED NUMERIC PATTERN GENERATOR =====")

try:
    rows = int(input("Enter the number of rows (1-10): "))

    if rows < 1 or rows > 10:
        print("Invalid input! Enter a number from 1 to 10.")

    else:
        # Pattern 1: Increasing numbers
        print("\nPattern 1: Increasing Numbers")

        for i in range(1, rows + 1):
            for j in range(1, i + 1):
                print(j, end=" ")
            print()

        # Pattern 2: Repeated row numbers
        print("\nPattern 2: Repeated Row Numbers")

        for i in range(1, rows + 1):
            for j in range(1, i + 1):
                print(i, end=" ")
            print()

        # Pattern 3: Inverted numbers
        print("\nPattern 3: Inverted Number Pattern")

        for i in range(rows, 0, -1):
            for j in range(1, i + 1):
                print(j, end=" ")
            print()

except ValueError:
    print("Invalid input! Please enter a whole number.")

print("\nPattern generation completed.")
```

26°C
Partly cloudy
Search

```
======= RESTART: C:/Users/YOGITHA/OneDrive/Desktop/7.Advanced numer
===== ADVANCED NUMERIC PATTERN GENERATOR =====
Enter the number of rows (1-10): 4

Pattern 1: Increasing Numbers
1
1 2
1 2 3
1 2 3 4

Pattern 2: Repeated Row Numbers
1
2 2
3 3 3
4 4 4 4

Pattern 3: Inverted Number Pattern
1 2 3 4
1 2 3
1 2
1

Pattern generation completed.
```

10. Conclusion

The Advanced Numeric Pattern Generator demonstrates how Python can generate different numeric patterns using nested loops. It helps students understand loop control, the `range()` function, conditional statements, and the use of variables in solving programming problems.